
KICSIT Management System

Learning Management System (LMS) — C# .NET Console Application

Technical Documentation & Docker Deployment Report

Project Name	KICSIT Learning Management System
Technology	.NET 10 / C# Console Application
Database	MongoDB (Hybrid: MongoDB + Text Files)
UI Library	Spectre.Console
Containerization	Docker & Docker Compose
Repository	github.com/Ayesha-Noor-1/Learning-Management-System-C-
Report Date	April 20, 2026
Developer	Ayesha Noor

Department of Computer Science — Institute Report

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Project Overview.....	4
1.2 Motivation and Scope	4
2. System Requirements	5
2.1 Software Prerequisites.....	5
2.2 Hardware Recommendations	5
3. System Architecture	6
3.1 Architectural Overview	6
3.2 Project Layout.....	6
4. Object-Oriented Design	7
4.1 Abstraction	7
4.2 Encapsulation	7
4.3 Inheritance.....	7
4.4 Polymorphism.....	7
4.5 Template Method Pattern	7
4.6 Virtual Methods and Sealed Classes	7
4.7 Composition and Method Overloading.....	7
5. Feature Modules	9
5.1 Guest / Main Menu	9
5.2 Student Portal	9
5.3 Teacher Portal.....	9
5.4 Admin Portal	9
6. Hybrid Data Storage Strategy	10
6.1 MongoDB Collections	10
6.2 Text File Storage.....	10
6.3 Database Seeding and Initialisation.....	10
7. Application Screenshots — Running System.....	11
7.1 Guest Main Menu.....	11
7.2 Faculty Information	11
7.3 Sign-In Role Selection	12
7.4 Credential Entry	13
7.5 Student Portal — Dashboard.....	13
7.6 Student Attendance View	14
7.7 Student — Download Quiz Results (CSV).....	14
7.8 Teacher Portal — Mark Attendance	15
7.9 Teacher Portal — Download All Quiz Results (CSV)	15

7.10 UI Design Summary	16
8. Configuration	17
8.1 appsettings.json	17
8.2 Environment Variable Override	17
8.3 MongoDB Atlas Configuration	17
9. Docker Implementation	18
9.1 Overview	18
9.2 Dockerfile Analysis	18
9.3 docker-compose.yml Services	18
9.4 Deployment Commands	18
10. Docker Implementation — Screenshots	19
10.1 Docker Build & Container Lifecycle (Command Prompt)	19
10.2 Docker Desktop — Container Logs (MongoDB Connection Error)	19
10.3 Docker Desktop — Images View	20
10.4 Running the Container — Standalone docker run	20
10.5 Docker Desktop — Container Running Successfully	21
11. Troubleshooting Guide	23
12. Testing	24
12.1 Unit Testing with xUnit	24
12.2 Running Tests	24
12.3 Test Coverage Areas	24
13. Security Considerations	25
13.1 Password Hashing	25
13.2 Connection String Management	25
13.3 Sealed Classes	25
13.4 Network Access	25
14. Conclusion and Future Enhancements	26
14.1 Summary	26
14.2 Potential Future Enhancements	26
14.3 Repository	26

1. Introduction

The KICSIT Management System is a feature-rich .NET console application developed in C# that serves as a comprehensive Learning Management System (LMS) for a small educational institute. The system provides structured portals for students, teachers, and administrators, backed by a hybrid data persistence strategy combining MongoDB and traditional text file I/O.

This report documents the full system architecture, object-oriented design principles applied, module descriptions, data storage strategy, UI screenshots of the running application, and the Docker-based containerization and deployment pipeline.

1.1 Project Overview

The project simulates the information management needs of KICSIT (a representative small institute). It exposes:

- A public-facing guest menu with faculty listings, course information, and admission details.
- A Student Portal for attendance, notices, timetable viewing, quiz attempts, CSV result export, and personal results.
- A Teacher Portal for timetable access, quiz authoring, student results management, attendance marking, and CSV export.
- An Admin Portal for roster management, notice publishing, timetable editing, teacher administration, and system statistics.

1.2 Motivation and Scope

The project demonstrates practical application of Object-Oriented Programming (OOP) principles — abstraction, encapsulation, inheritance, and polymorphism — within a real-world domain. It also showcases hybrid persistence (MongoDB + file system), modern console UI design with Spectre.Console, and containerised deployment using Docker.

2. System Requirements

2.1 Software Prerequisites

Component	Requirement	Notes
.NET SDK	.NET 10 (SDK 10.x)	Match TargetFramework in .csproj
MongoDB	Atlas	Atlas URI
Visual Studio 2022	Recommended IDE	Any editor with dotnet CLI works
Docker Desktop	v4.69.0 (as used)	For containerised deployment
OS	Windows (primary)	Linux/macOS via Docker

2.2 Hardware Recommendations

- Minimum 4 GB RAM (8 GB recommended for Docker + MongoDB)
- At least 2 GB free disk space for Docker images and MongoDB data volumes
- Internet connection for MongoDB Atlas or Docker image pulls

3. System Architecture

3.1 Architectural Overview

The KICSIT LMS follows a layered console application architecture. The presentation layer is handled by Spectre.Console, business logic resides in distinct portal classes (Student, Teacher, Admin), and data access is abstracted through repositories and the LmsDatabase initializer. The application also mixes two storage backends — MongoDB for structured entities and plain text files for line-based content.

3.2 Project Layout

File / Folder	Purpose
Dockerfile	Multi-stage build: SDK stage compiles, runtime stage runs the app
docker-compose.yml	Orchestrates MongoDB 7 service + LMS application container
LMS-C#/Program.cs	Entry point — initialises file system, connects to MongoDB
LMS-C#/AppPaths.cs	All path constants for Documents\KICSITData (file-backed data)
LMS-C#/Oop/	Interfaces and abstract base classes (IPortal, PortalBase, etc.)
LMS-C#/Data/	MongoDB access layer: LmsDatabase, repositories, initializer
LMS-C#/Models/	BSON document models for MongoDB collections
LMS-C#/appsettings.json	Connection string and database name configuration
LMS-C#/GeneralView.cs	Landing / guest menu implementation
LMS-C#/AuthManager.cs	Sign-in flow (reads from MongoDB)
LMS-C#/Student.cs	Student portal class (attendance, quiz, notices, timetable, CSV export)
LMS-C#/Teacher.cs	Teacher portal class (quiz authoring, results, attendance, CSV export)
LMS-C#/Admin.cs	Admin portal class (roster, notices, stats, timetable)
LMS-C#/UI.cs	Spectre.Console presentation helpers (tables, menus, panels)
LMS-C#/*.txt	Seed data files (seeded to MongoDB on first empty-DB run)
LMS-C#/LMS-C#.Tests/	xUnit unit tests (separate .csproj with friend assembly access)
LMS-C#.slnx	Visual Studio solution file

4. Object-Oriented Design

The LMS applies all four pillars of OOP rigorously. Each concept is mapped to a concrete class or interface in the codebase.

4.1 Abstraction

Three interfaces define the contracts used throughout the system:

- `IPortal` — Any signed-in area (student, teacher, admin) implements this interface. Callers depend on the contract, not the concrete type.
- `ILoginValidator` — Abstracts the credential checking logic. The concrete `MongoCredentialValidator` reads hashed passwords from MongoDB.
- `IQuizResultWriter` — Abstracts quiz result persistence, supporting both MongoDB and legacy file output (`FileQuizResultWriter`).

4.2 Encapsulation

`PortalBase` keeps `_username` as a private field, exposing only `DisplayUsername` publicly. Validators and writers hide all storage details behind their interfaces. Each portal class owns its data access calls without leaking implementation specifics to callers.

4.3 Inheritance

`Student`, `Teacher`, and `Admin` all inherit from the abstract `PortalBase` class. `PortalBase` provides shared behaviour — the main menu loop, welcome display, role header rendering — while each subclass overrides the abstract hooks to supply role-specific menus and action handling.

4.4 Polymorphism

`AuthManager` receives an `IPortal` reference after sign-in and calls `Run()` without knowing whether it is a `Student`, `Teacher`, or `Admin` object. `ILoginValidator` is implemented by `MongoCredentialValidator`. `IQuizResultWriter` is implemented by both `FileQuizResultWriter` and the MongoDB-backed writer.

4.5 Template Method Pattern

`PortalBase.Run()` defines the menu loop skeleton. Subclasses override abstract hooks: `DrawRoleHeader`, `BuildMenuPrompt`, and `ExecuteSelection`. This ensures the loop structure is consistent across all portals while customisation points are cleanly separated.

4.6 Virtual Methods and Sealed Classes

`PortalBase` exposes `BeforeMenuIteration` and `WriteWelcomeLine` as virtual so subclasses may optionally override them. `Student`, `Teacher`, `Admin`, `MongoCredentialValidator`, and `FileQuizResultWriter` are all sealed to prevent unintended further subclassing.

4.7 Composition and Method Overloading

Portals compose QuizAttemptRepository and other static repositories for persistence, keeping UI concerns in UI helpers. InputValidation demonstrates method overloading: IsMeaningful(string?) vs IsMeaningful(string?, int maxLength).

5. Feature Modules

5.1 Guest / Main Menu

The landing screen (GeneralView.cs) presents publicly accessible information without requiring login:

- Faculty directory listing with names, departments, and email addresses.
- Courses offered by the institute.
- Admission information and procedures.
- Sign-in entry point routing to the role-specific portal.

5.2 Student Portal

Feature	Description
Attendance	View attendance records read from attendanceCS-1A.txt (colour-coded P/A)
Notice Board	Read institute notices stored in MongoDB notices collection
Timetable	View class schedule from timetableCS-1A.txt
Take Quiz	Interactive MCQ quiz; attempts stored in MongoDB quizAttempts collection
My Quiz Results	View personal quiz attempts with marks, duration, and timestamp
Download Results (CSV)	Export personal quiz results to timestamped CSV file in KICSITData folder

5.3 Teacher Portal

- Timetable — View teaching schedule.
- Quiz Authoring — Create and manage MCQ questions written to quizmakeCS1-A.txt.
- All Student Results — View all quiz attempts from the MongoDB quizAttempts collection as a Spectre table.
- Download Quiz Results (CSV) — Export all student attempts to a timestamped CSV in KICSITData.
- Manage Quiz & Results — Clear MCQ file and/or clear MongoDB quiz attempts.
- Mark Attendance — Select Present or Absent for each student by name using an arrow-key menu.
- View Attendance — Review current attendance records.

5.4 Admin Portal

- Add / Remove Students — Roster management via MongoDB students collection.
 - Notices — Publish and manage institute-wide notices.
 - Add Teachers — Create teacher accounts with MongoDB credentials.
 - Timetable Editing — Modify timetable entries in the text file.
 - Statistics — Spectre-rendered summary table with counts from MongoDB.
 - Return to Main Menu — Logout and return to the guest landing screen.
-

6. Hybrid Data Storage Strategy

The system intentionally uses two storage backends to demonstrate both a database layer and traditional file I/O within a single project.

6.1 MongoDB Collections

Collection	Content
users	User accounts with hashed passwords (students, teachers, admins)
students	Student roster fields linked to login user documents
faculty	Faculty directory (name, department, email) linked to teacher login
notices	Institute notices published by admin
quizAttempts	Per-user quiz attempt records (marks, duration, timestamp)
courses	Course listings shown on the main menu
admission	Admission information text shown on the main menu

6.2 Text File Storage

File	Content
timetableCS-1A.txt	Class timetable (line-based, editable by admin)
attendanceCS-1A.txt	Attendance records for the class
quizmakeCS1-A.txt	MCQ questions authored by teachers
quizresult.txt	Legacy quiz result file (optional teacher view)

6.3 Database Seeding and Initialisation

1. Creates the KICSITData folder under %USERPROFILE%\Documents if it does not exist.
 2. Copies missing seed .txt files from the build output to KICSITData.
 3. Connects to MongoDB using the connection string from appsettings.json.
 4. Seeds MongoDB collections from seed text files only when collections are empty.
 5. After first seeding, sign-in and admin changes use MongoDB exclusively.
-

7. Application Screenshots — Running System

The following screenshots were captured from the live running KICSIT Management System on April 20, 2026. They document the complete user journey from the guest main menu through student and teacher portal interactions.

7.1 Guest Main Menu

The application launches to the KICSIT Management System main menu. Spectre.Console renders an arrow-key navigable selection with five options: Faculty information, Available courses, Admission portal, Log in, and Exit.

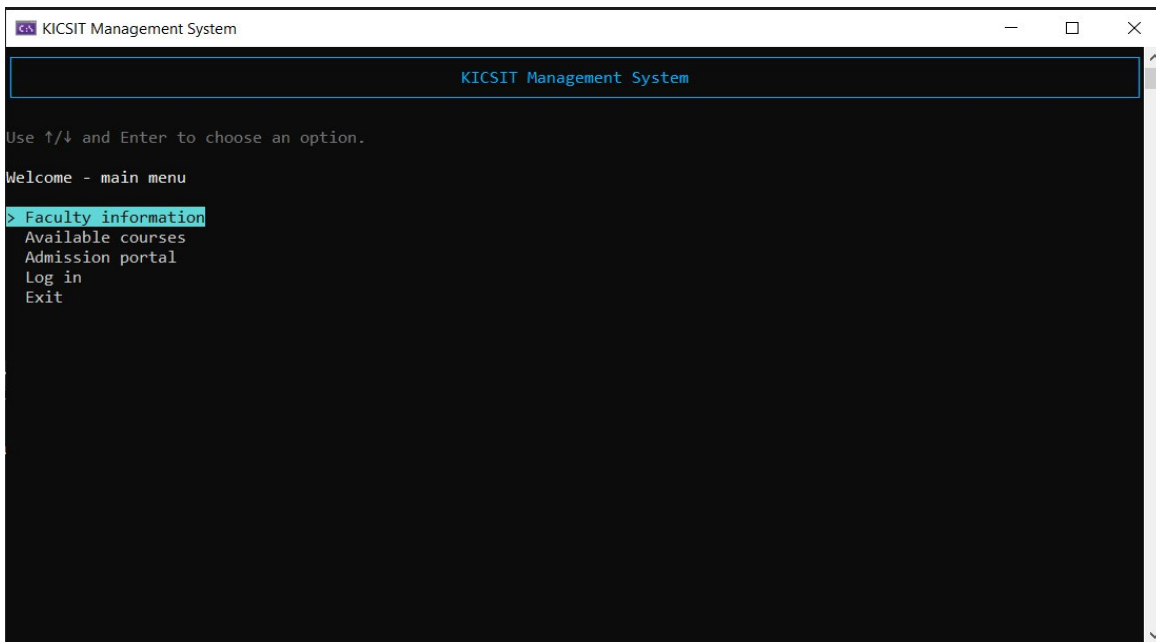


Figure 1: KICSIT Management System — Guest Main Menu (arrow-key navigation with Spectre.Console)

7.2 Faculty Information

Selecting 'Faculty information' displays a Spectre.Console formatted table showing all faculty members: their name, department, and institute email. The prompt at the bottom offers return to the main menu or application exit.

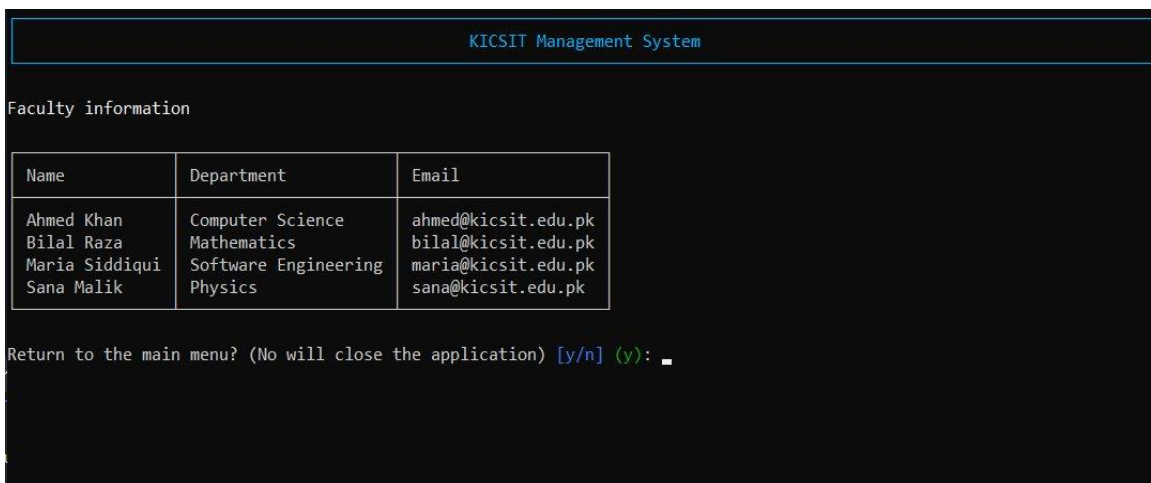


Figure 2: Faculty Information table — Name, Department, Email for four faculty members

7.3 Sign-In Role Selection

Choosing 'Log in' from the main menu presents a role selection screen. The user selects between Student, Teacher, and Admin using arrow keys. This routes to the appropriate portal after credential verification.

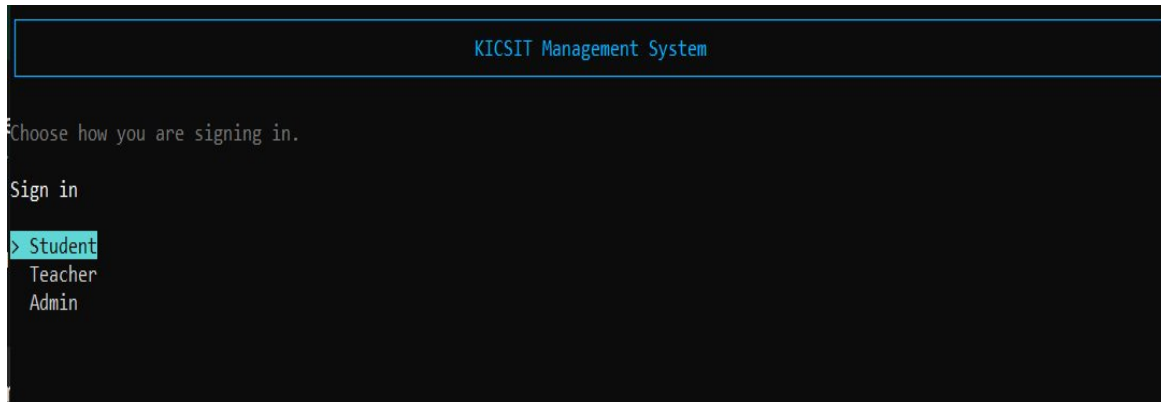


Figure 3: Sign-In screen — Role selection prompt (Student, Teacher, Admin)

7.4 Credential Entry

After selecting a role, the system prompts for Name and Password. The password field is masked (shown as asterisks *****) using Spectre.Console's TextPrompt with secret mode. Credentials are verified against hashed values in MongoDB.

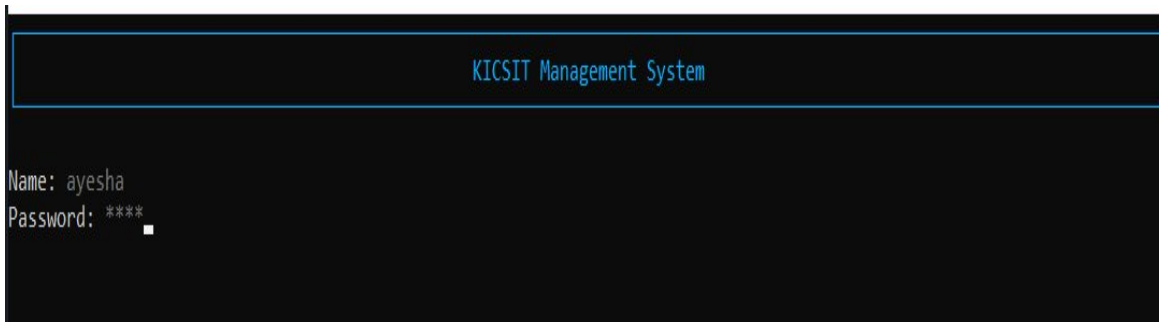


Figure 4: Credential entry — Name visible, password masked with asterisks for security

7.5 Student Portal — Dashboard

Upon successful login, the Student Portal opens with a green-accented header ('Student Portal') and a personalised welcome message ('Welcome, AYESHA'). The menu offers: Attendance, Notice board, Timetable, Take quiz, My quiz results, Download my quiz results (CSV), and Logout.

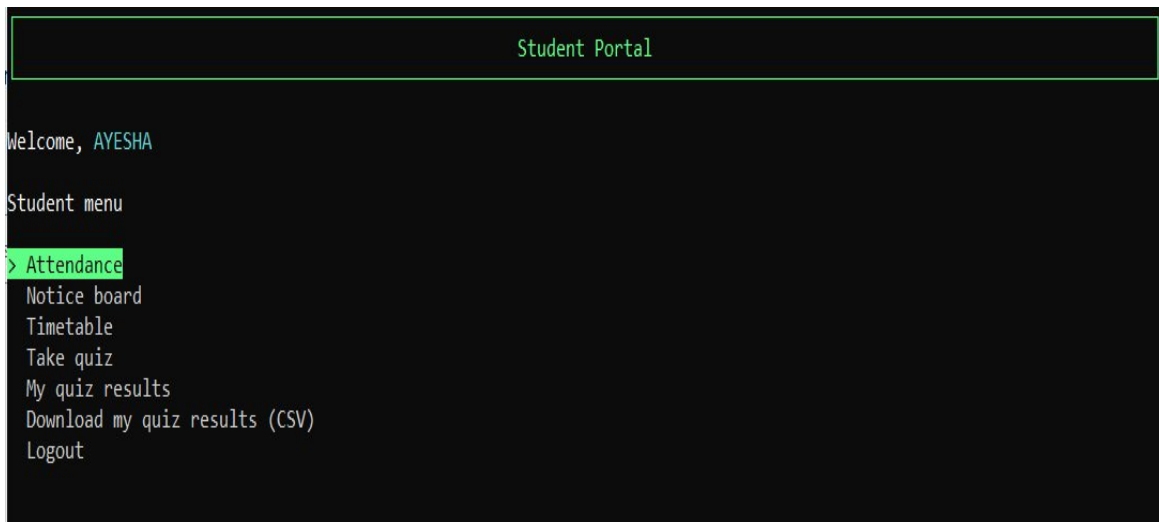
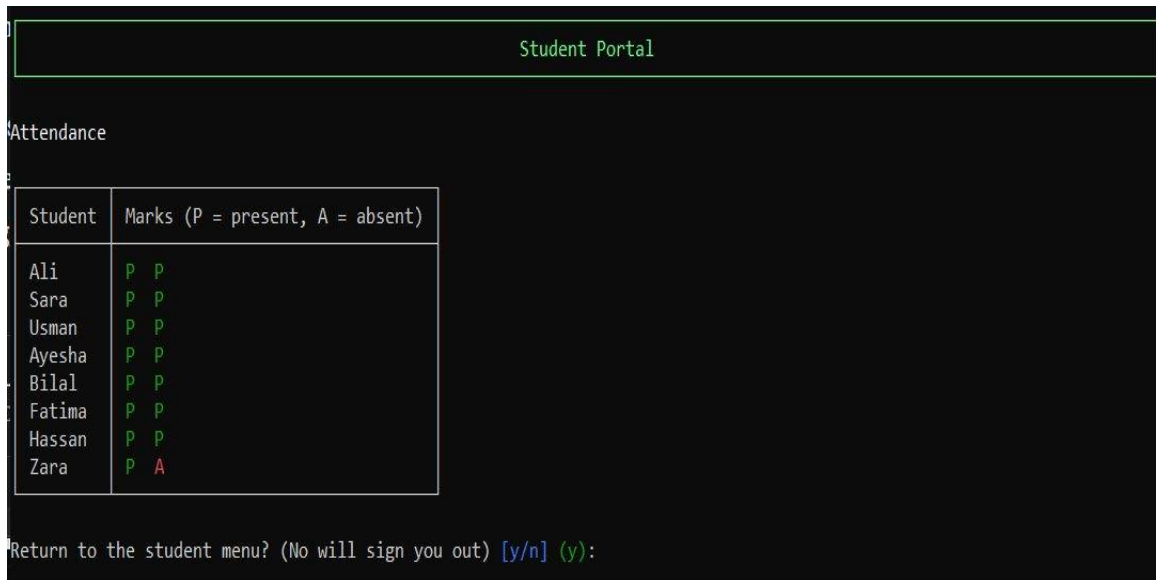


Figure 5: Student Portal dashboard — Welcome message and full student menu

7.6 Student Attendance View

The Attendance screen displays a table of all students with their attendance marks colour-coded: P (Present) in green and A (Absent) in red. This data is read from the attendanceCS-1A.txt file in the KICSITData folder. The last student (Zara) shows one absence marked in red.



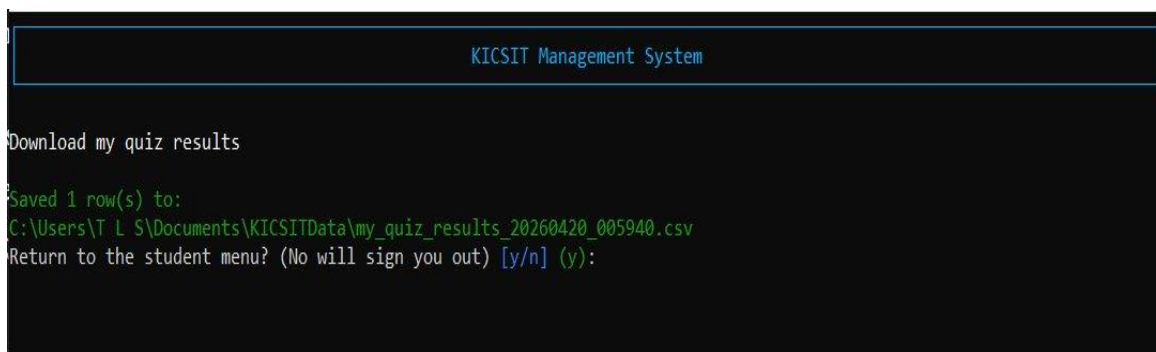
The screenshot shows a terminal window titled "Student Portal". Below the title bar, the word "Attendance" is displayed. A table with two columns, "Student" and "Marks (P = present, A = absent)", lists eight students: Ali, Sara, Usman, Ayesha, Bilal, Fatima, Hassan, and Zara. Each student has two marks, both of which are green 'P' for all students except Zara, who has one green 'P' and one red 'A'. At the bottom of the terminal, a prompt asks "Return to the student menu? (No will sign you out) [y/n] (y):".

Student	Marks (P = present, A = absent)
Ali	P P
Sara	P P
Usman	P P
Ayesha	P P
Bilal	P P
Fatima	P P
Hassan	P P
Zara	P A

Figure 6: Attendance table — Colour-coded Present (green P) and Absent (red A) marks for all students

7.7 Student — Download Quiz Results (CSV)

The 'Download my quiz results (CSV)' option exports the student's personal quiz attempts to a timestamped CSV file. The system confirms the export by displaying the number of rows saved and the full file path: C:\Users\T L S\Documents\KICSITData\my_quiz_results_20260420_005940.csv



The screenshot shows a terminal window titled "KICSIT Management System". The prompt "Download my quiz results" has been entered. The system responds with "Saved 1 row(s) to:" followed by the file path "C:\Users\T L S\Documents\KICSITData\my_quiz_results_20260420_005940.csv" in green text. At the bottom, a prompt asks "Return to the student menu? (No will sign you out) [y/n] (y):".

Figure 7: Student CSV export — Saved 1 row to timestamped CSV file in KICSITData folder

7.8 Teacher Portal — Mark Attendance

In the Teacher Portal, the 'Mark attendance' feature lists each student by name sequentially. For each student, the teacher selects Present or Absent from an arrow-key menu rendered in the blue-accented Teacher Portal theme. The screenshot shows the teacher marking attendance for Ali, Sara, and Usman, with the Attendance prompt showing 'Present' currently highlighted.



Figure 8: Teacher Portal — Mark attendance with per-student Present/Absent selection menu

7.9 Teacher Portal — Download All Quiz Results (CSV)

The teacher can download all students' quiz attempts as a CSV file. The export confirms the row count and saves to a timestamped file: C:\Users\T L S\Documents\KICSITData\quiz_attempts_all_20260420_010058.csv. This file contains results from all students across all quiz attempts.

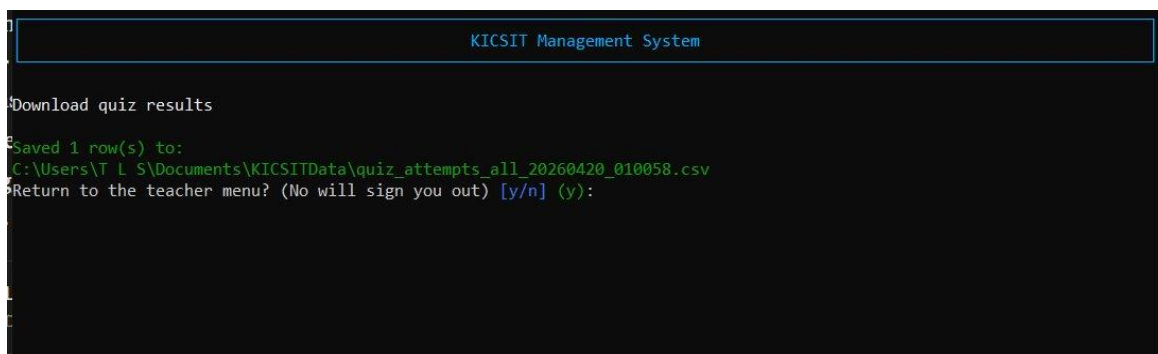


Figure 9: Teacher Portal CSV export — All quiz attempts exported to timestamped CSV in KICSITData

7.10 UI Design Summary

Portal	Accent Colour	Key UI Elements Shown
Guest / Main	Blue (cyan)	Arrow-key SelectionPrompt, header panel, return prompt
Student Portal	Green	Welcome banner, full menu, attendance colour table, CSV confirm
Teacher Portal	Purple/Blue	Per-student attendance loop, CSV export confirmation
All Portals	Masked input	Password entry with ***** masking via Spectre TextPrompt

8. Configuration

8.1 appsettings.json

The primary configuration file is LMS-C#/appsettings.json, which is copied to the build output directory at compile time.

8.2 Environment Variable Override

- MONGODB_CONNECTION_STRING — overrides MongoDB:ConnectionString
- MONGODB_DATABASE_NAME — overrides MongoDB:DatabaseName

8.3 MongoDB Atlas Configuration

6. Create a database user under Security > Database Access.
7. Add your IP address to Security > Network Access.
8. Copy the connection string from Clusters > Connect > Drivers (C#).
9. Replace password in URI; URL-encode special characters if needed.
10. Set MongoDB:ConnectionString in appsettings.json or as environment variable.

9. Docker Implementation

9.1 Overview

The application is fully containerised using Docker with a multi-stage Dockerfile and a docker-compose.yml that orchestrates both the LMS application and a MongoDB 7 database service. This enables the system to run on any machine with Docker installed, without requiring .NET SDK or MongoDB to be installed natively.

9.2 Dockerfile Analysis

Stage 1 — Build Stage (SDK Image)

- Base image: mcr.microsoft.com/dotnet/sdk:10
- WORKDIR /src/LMS-C# — sets the source working directory
- COPY LMS-C#/LMS-C#.csproj . — copies project file first (layer cache)
- RUN dotnet restore — restores NuGet packages
- COPY LMS-C#/. — copies full source files
- RUN dotnet publish -c Release -o /app/publish --no-restore

Stage 2 — Runtime Stage (Runtime-Only Image)

- Base image: mcr.microsoft.com/dotnet/runtime:10 (no SDK, smaller image)
- WORKDIR /app — runtime working directory
- COPY --from=build /app/publish . — copies compiled output from build stage
- ENTRYPOINT ["dotnet", "LMS-C#.dll"]

9.3 docker-compose.yml Services

Service	Image	Configuration
mongo	mongo:7	Volume: mongo_data, internal network, health check
lms	Built from Dockerfile	Depends on mongo, TTY required, env vars for connection, kicsit_data volume

9.4 Deployment Commands

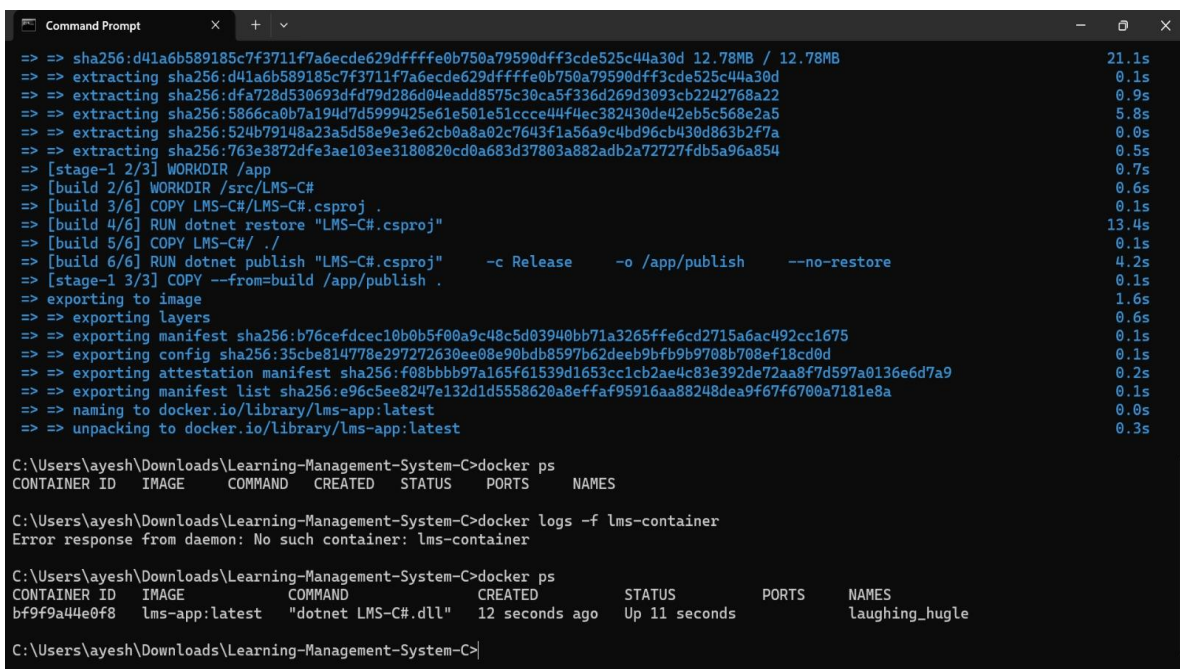
Command	Purpose
docker compose up -d mongo	Start MongoDB in background
docker compose run --rm -it --build lms	Build and run LMS interactively (required)
docker build -t lms-app .	Build image only
docker run -d -p 8080:8080 --name lms-container lms-app	Run container standalone
docker logs -f lms-container	Stream container logs
docker compose down	Stop all services
docker compose down -v	Stop and wipe data volumes

10. Docker Implementation — Screenshots

The following screenshots document the actual Docker build and deployment process captured during the project demonstration on April 20, 2026.

10.1 Docker Build & Container Lifecycle (Command Prompt)

This screenshot shows the complete multi-stage Docker build output. All six build steps complete successfully, layers are exported, and the image is tagged as `docker.io/library/lms-app:latest`. A second `docker ps` confirms the running container.



```
C:\Users\ayesh\Downloads\Learning-Management-System-C>docker build -t lms-app:latest .
=> sha256:d41a6b589185c7f3711f7a6ecde629dffffe0b750a79590dff3cde525c44a30d 12.78MB / 12.78MB 21.1s
=> extracting sha256:d41a6b589185c7f3711f7a6ecde629dffffe0b750a79590dff3cde525c44a30d 0.1s
=> extracting sha256:d41a6b589185c7f3711f7a6ecde629dffffe0b750a79590dff3cde525c44a30d 0.9s
=> extracting sha256:d41a6b589185c7f3711f7a6ecde629dffffe0b750a79590dff3cde525c44a30d 5.8s
=> extracting sha256:5866ca0b7a194d7d5999425e61e501e51ccce44f4ec382430de42eb5c568e2a5 0.0s
=> extracting sha256:524b79148a23a5d58e9e3e62cb0a8a02c7643f1a56a9c4bd96cb430d863b2f7a 0.5s
=> extracting sha256:763e3872dfe3ae103ee3180820cd0a683d37803a882adb2a72727fdb5a96a854 0.7s
=> [stage-1 2/3] WORKDIR /app 0.6s
=> [build 2/6] WORKDIR /src/LMS-C# 0.1s
=> [build 3/6] COPY LMS-C#/LMS-C#.csproj . 13.4s
=> [build 4/6] RUN dotnet restore "LMS-C#.csproj" 0.1s
=> [build 5/6] COPY LMS-C#/ ./ 4.2s
=> [build 6/6] RUN dotnet publish "LMS-C#.csproj" -c Release -o /app/publish --no-restore 0.1s
=> [stage-1 3/3] COPY --from=build /app/publish . 1.6s
=> exporting to image 0.6s
=> exporting layers 0.1s
=> exporting manifest sha256:b76cefdceec10b0b5f00a9c48c5d03940bb71a3265ffe6cd2715a6ac492cc1675 0.1s
=> exporting config sha256:35cbe814778e297272630ee08e90b0db8597b62deeb9bf9b9708b708ef18cd0d 0.1s
=> exporting attestation manifest sha256:f08bbbb97a165f61539d1653cc1cb2ae4c83e392de72aa8f7d597a0136e6d7a9 0.2s
=> exporting manifest list sha256:e96c5ee8247e132d1d5558620a8effaf95916aa88248dea9f67f6700a7181e8a 0.1s
=> naming to docker.io/library/lms-app:latest 0.0s
=> unpacking to docker.io/library/lms-app:latest 0.3s

C:\Users\ayesh\Downloads\Learning-Management-System-C>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
C:\Users\ayesh\Downloads\Learning-Management-System-C>docker logs -f lms-container
Error response from daemon: No such container: lms-container

C:\Users\ayesh\Downloads\Learning-Management-System-C>docker ps
CONTAINER ID   IMAGE      COMMAND                  STATUS        PORTS        NAMES
bf9f9a44e0f8   lms-app:latest  "dotnet LMS-C#.dll"      Up 11 seconds  laughing_hugle

C:\Users\ayesh\Downloads\Learning-Management-System-C>
```

Figure 10: Docker build output and container listing in Command Prompt

10.2 Docker Desktop — Container Logs (MongoDB Connection Error)

Container `laughing_hugle` exited with code 139. Logs reveal the app detected MongoDB was unreachable, printed a structured diagnostic message, and attempted `Console.ReadKey()` without a TTY — causing the `InvalidOperationException`. The correct fix is `docker compose run --rm -it --build lms`.

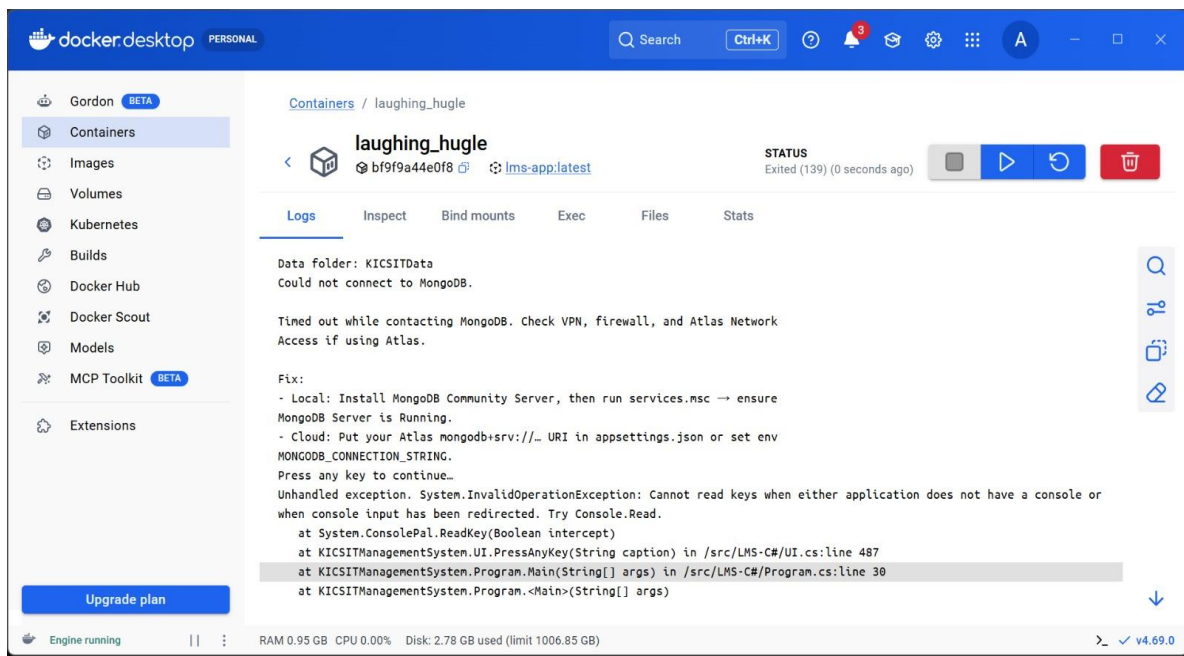


Figure 11: Docker Desktop container logs — MongoDB timeout and TTY error

10.3 Docker Desktop — Images View

Docker Desktop confirms the `lms-app:latest` image (308.39 MB) was built and stored locally with image ID `e96c5ee8247e`.

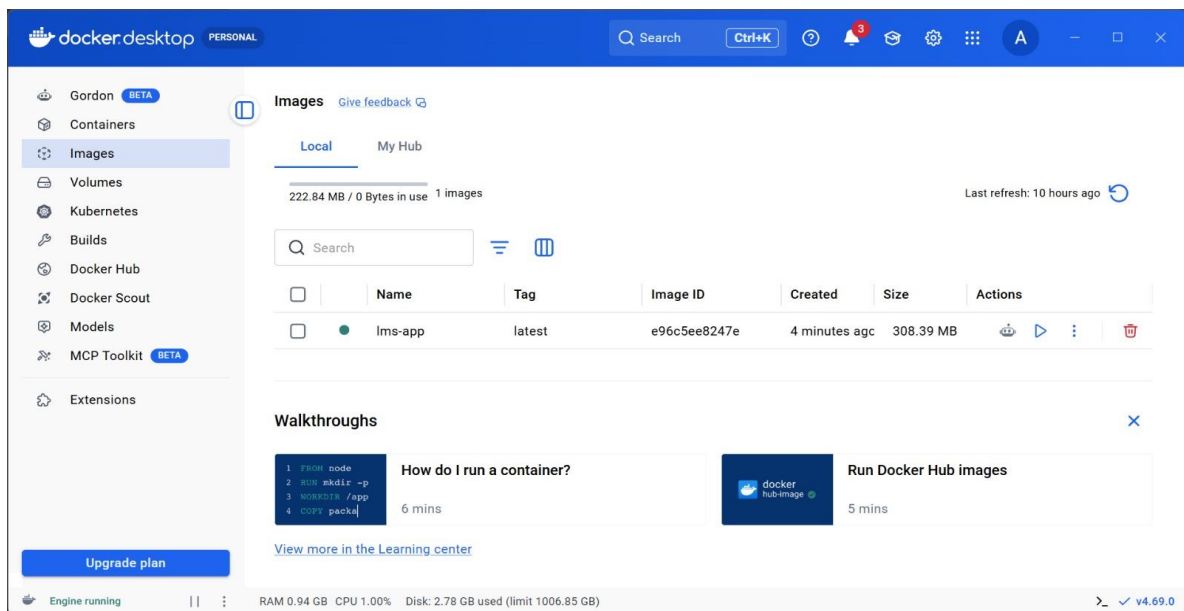


Figure 12: Docker Desktop Images view — `lms-app:latest` (308.39 MB)

10.4 Running the Container — Standalone docker run

The `docker run -d` command creates a new container (`lms-container`) and returns the container ID. The previously running container (`laughing_hugle`) is also visible in `docker ps`, confirming both containers existed simultaneously.

```
Command Prompt
=> sha256:d41a6b589185c7f3711f7a6ecde629dffffe0b750a79590dfff3cde525c44a30d 12.78MB / 12.78MB 21.1s
=> extracting sha256:d41a6b589185c7f3711f7a6ecde629dffffe0b750a79590dfff3cde525c44a30d 0.1s
=> extracting sha256:dfa728d530693dfd79d286d04eadd8575c30ca5f336d269d3093cb2242768a22 0.9s
=> extracting sha256:5866ca0b7a194d7d5999425e61e501e51ccce44f4ec382430de42eb5c568e2a5 5.8s
=> extracting sha256:524b79148a23a5d58e9e3e62cb0a8a02c7643f1a56a9c4bd96c6b430d863b2f7a 0.0s
=> extracting sha256:763e3872dfe3ae103ee3180820cd0a683d37803a882adb2a72727fdb5a96a854 0.5s
=> [stage-1 2/3] WORKDIR /app 0.7s
=> [build 2/6] WORKDIR /src/LMS-C# 0.6s
=> [build 3/6] COPY LMS-C#/LMS-C#.csproj . 0.1s
=> [build 4/6] RUN dotnet restore "LMS-C#.csproj" 13.4s
=> [build 5/6] COPY LMS-C#/ / 0.1s
=> [build 6/6] RUN dotnet publish "LMS-C#.csproj" -c Release -o /app/publish --no-restore 4.2s
=> [stage-1 3/3] COPY --from=build /app/publish . 0.1s
=> exporting to image 1.6s
=> exporting layers 0.6s
=> exporting manifest sha256:b76cefdcec10b0b5f0a9c48c5d03940bb71a3265ffe6cd2715a6ac492cc1675 0.1s
=> exporting config sha256:35cbe814778e297272630ee08e90b0db8597b62deeb9bfb9b9708b708ef18cd8d 0.1s
=> exporting attestation manifest sha256:f08bbb97a165f61539d1653c1cb2ae4c83e392de72aa8f7d597a0136e6d7a9 0.2s
=> exporting manifest list sha256:e96c5ee8247e132d1d5558620a8effaf95916aa88248dea9f67f6708a7181e8a 0.1s
=> naming to docker.io/library/lms-app:latest 0.0s
=> unpacking to docker.io/library/lms-app:latest 0.3s

C:\Users\ayesh\Downloads\Learning-Management-System-C>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
bf9f9a44e9f8 lms-app:latest "dotnet LMS-C#.dll" 12 seconds ago Up 11 seconds laughing_hugle

C:\Users\ayesh\Downloads\Learning-Management-System-C>docker logs -f lms-container
Error response from daemon: No such container: lms-container

C:\Users\ayesh\Downloads\Learning-Management-System-C>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
bf9f9a44e9f8 lms-app:latest "dotnet LMS-C#.dll" 12 seconds ago Up 11 seconds laughing_hugle

C:\Users\ayesh\Downloads\Learning-Management-System-C>docker run -d -p 8080:8080 --name lms-container lms-app
98082bce9bccc991b31dca676257fba6724bf3bc07a14b6973df4d08a21ba84

C:\Users\ayesh\Downloads\Learning-Management-System-C>
```

Figure 13: docker run creating lms-container, confirmed in docker ps output

10.5 Docker Desktop — Container Running Successfully

The following screenshots confirm the lms-container running successfully in Docker Desktop. The container (ID: 9bcf8abc01a2) is shown with status Running, consuming 19.74% CPU and 45.43 MB RAM. The container logs display the KICSIT Management System main menu with all navigation options rendered correctly, and the application terminal view confirms the interactive menu is fully operational.

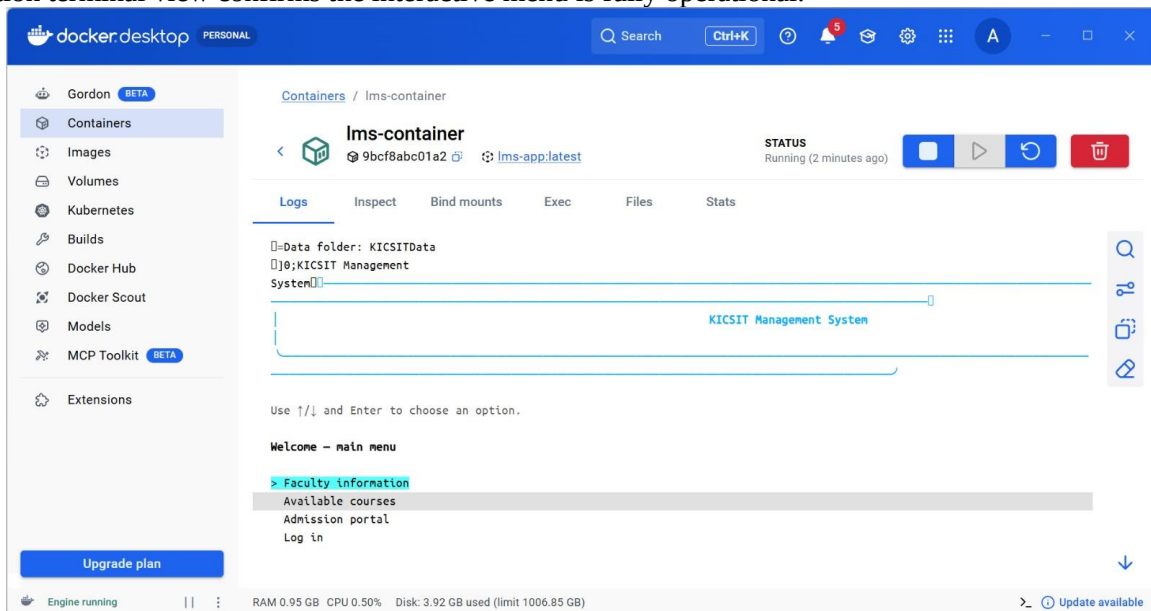


Figure 14: Docker Desktop Containers view — lms-container running (ID: 9bcf8abc01a2, CPU: 19.74%)

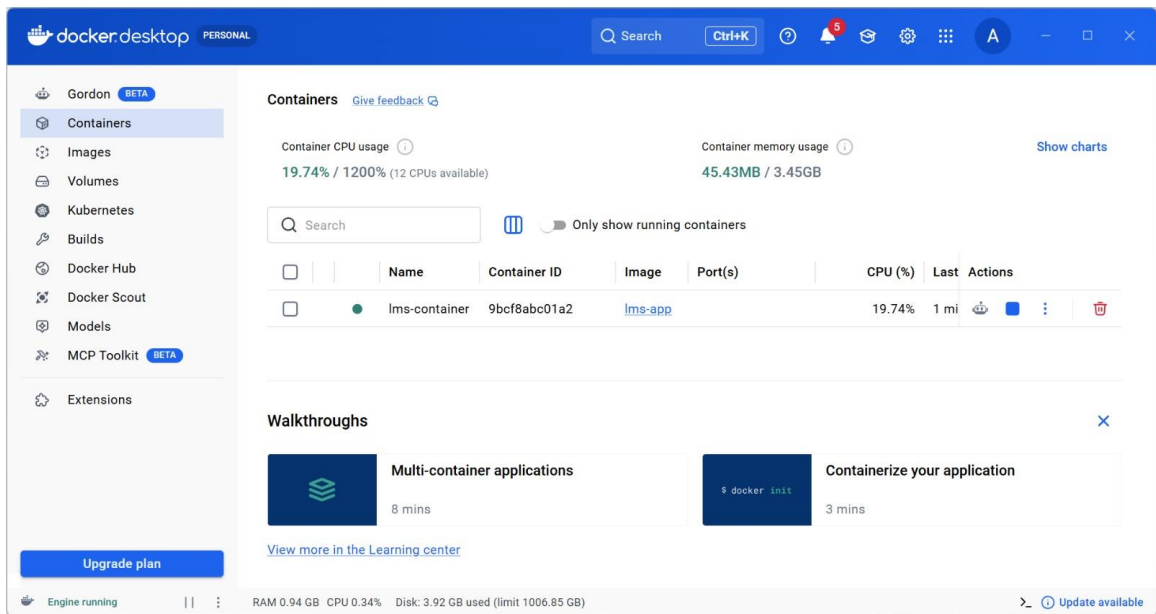


Figure 15: Docker Desktop container logs — KICSIT Management System main menu rendering in container

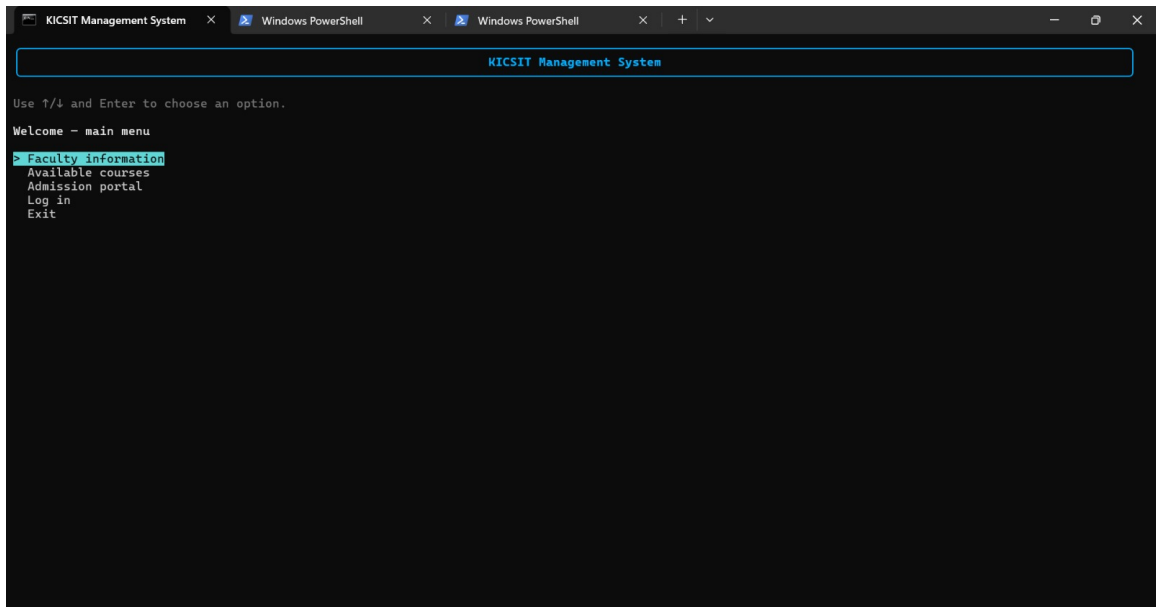


Figure 16: Application terminal — KICSIT Management System interactive menu fully operational inside Docker container

11. Troubleshooting Guide

Issue	Cause	Resolution
Build error: cannot copy LMS-C#.exe	App is running and locking the output .exe	Stop debugging (Shift+F5), close console, rebuild
Cannot connect to MongoDB	mongod not running or Atlas IP not whitelisted	Start MongoDB service OR add IP in Atlas Network Access
bad auth: authentication failed	Using Atlas website email not database user	Use database user from Atlas Database Access
Sign-in fails after adding users	New accounts in MongoDB, not login*.txt	Ensure MongoDB running; restart app to re-seed if DB reset
Console menus broken in Docker	No pseudo-TTY (-it flag missing)	Use docker compose run --rm -it --build lms
Container exits immediately	MongoDB unreachable + no TTY	Use Compose; always add -it flag

12. Testing

12.1 Unit Testing with xUnit

- Framework: xUnit — open-source unit testing framework for .NET.
- Friend Assembly: InternalsVisibleTo allows test project to access internal classes.
- Separate .csproj: Test project has its own project file preventing test code from entering the production executable.

12.2 Running Tests

```
dotnet test LMS-C#.Tests\LMS-C#.Tests.csproj
```

12.3 Test Coverage Areas

- Input validation (InputValidation.IsMeaningful overloads)
 - Authentication flow logic (AuthManager)
 - Quiz attempt scoring and result recording
 - File path resolution (AppPaths constants)
-

13. Security Considerations

13.1 Password Hashing

All user passwords stored in MongoDB are hashed. The application never stores plain-text passwords. The MongoCredentialValidator performs hash comparison during authentication — as demonstrated in the credential entry screenshot where the password appears as ****.

13.2 Connection String Management

Sensitive MongoDB Atlas connection strings should not be committed to source control. The project supports environment variable override (MONGODB_CONNECTION_STRING). For Docker deployments, credentials are injected via docker-compose.yml or the -e flag.

13.3 Sealed Classes

Security-sensitive classes (MongoCredentialValidator, Admin, Teacher, Student, FileQuizResultWriter) are sealed, preventing unintended inheritance that could bypass authentication or data access controls.

13.4 Network Access

MongoDB Atlas Network Access should be restricted to known IPs in production. 0.0.0.0/0 is acceptable only for coursework demonstrations.

14. Conclusion and Future Enhancements

14.1 Summary

The KICSIT Learning Management System successfully demonstrates a production-quality C# .NET console application that applies all core OOP principles within a practical educational domain. The hybrid storage architecture (MongoDB + text files) fulfils the dual goal of showcasing both database-driven and file-based persistence. The Spectre.Console UI delivers a polished, navigable interface with role-specific colour schemes — as evidenced by the running system screenshots in Section 7.

The Docker implementation features a multi-stage Dockerfile and docker-compose.yml with MongoDB 7, enabling reproducible deployment without requiring .NET SDK or MongoDB to be installed natively. Screenshots in Sections 7 and 10 together document the full journey from running application to containerised deployment.

14.2 Potential Future Enhancements

- Web-based frontend using Blazor or ASP.NET Core for broader accessibility.
- REST API layer exposing LMS functionality to mobile or third-party clients.
- Role-based access control (RBAC) with finer-grained permissions.
- Email notifications for quiz results, notices, and attendance alerts.
- CI/CD pipeline (GitHub Actions) for automated build, test, and Docker image push.
- Grade book and assignment submission module extending the quiz system.
- Multi-class and multi-section support beyond the current CS-1A scope.

14.3 Repository

The full source code, Dockerfile, docker-compose.yml, and seed data files are available at:

<https://github.com/Ayesha-Noor-1/Learning-Management-System-C->

This project is submitted as coursework for the Department of Computer Science. No license file is included in the repository; reuse beyond coursework requires explicit permission from the author.
